MDMC
Master in Data Management and Curation

AREA SCIENCE PARK

SISSA

Scientific Programming Environment – System usage and scripting in Bash shell

Trieste, September 23rd 2025

- The Bourne Again Shell (bash)

- Started as part of the GNU Project to replace UNIX default shell (developed by Stephen Bourne at Bell Labs).
- Bash incorporates elements from the original /bin/sh, the Korn shell (ksh) and the C shell (csh).
- Bash is the default shell for most Linux distros and has been ported to other UNIX-like operating systems as well as to Microsoft Windows.

- The Bourne Again Shell (bash)

- Bash main features are:
  - A REPL (Read Evaluate Print Loop) interpreter with rich line editing and command history for interactive use.
  - Input/output redirection and piping.
  - An extensive set of flow control statement that allow a rich scripting experience.

- The Bourne Again Shell (bash)

- The GNU Project maintains an extensive manual page for bash: https://www.gnu.org/software/bash/manual/bash.html
- And when the Internet connection fails on you, there's the offline manual accessible by typing the following:

```
$ man bash
```

- Shell basics

- Listing the contents of the current working directory:

```
$ ls
```

- Long output listing with permissions ownership and mtimes:

```
$ ls -l
```

- Creating a new directory:

```
$ mkdir newdir
```

- Changing current working directory to newdir:

```
$ cd newdir
```

- Shell basics

- Moving to the parent of our current working directory:

```
$ cd ..
```

- Finding what type of file is newdir:

```
$ file newdir
newdir: directory
```

- Creating a new empty file:

```
$ touch newfile
```

- Shell basics

- Finding what type of file is newfile:

```
$ file newfile
newfile: empty
```

- Copying newfile to newcopy:

```
$ cp newfile newcopy
```

- Moving newfile inside newdir:

```
$ mv newfile ./newdir/
```

- Shell basics

- Rename newcopy to oldcopy:

```
$ mv newcopy oldcopy
```

- Create a hard link to oldcopy called newoldcopy:

```
$ ln oldcopy newoldcopy
```

- Create a symbolic link to oldcopy called newlink:

```
$ ln -s oldcopy newlink
```

- Shell basics

- Delete a file:

```
$ rm newdir/newfile
```

- Delete an empty directory:

```
$ rmdir newdir
```

- Recursively delete everything in the system as root (NEVER do this):

```
# rm -rf /
```

- Input/output redirection

- Each process launched in a textual shell receives 3 file-like objects already opened and ready to use:
  - standard input: read-only (usually the keyboard)
  - standard output: write-only (usually the console)
  - standard error: write-only (usually reserved for error messages)
- These three can be redirected by the shell itself to/from any other file and could also redirected to other processes via piping: the standard output of a process becomes the standard input of another process.
- Piping allows the concatenation of multiple filters (programs that take an input, modify it and output the result).

- Input/output redirection

- Printing "Hello World!" to the standard output:

```
$ echo "Hello World!"
```

- Redirecting the standard output to hello.txt:

```
$ echo "Hello World!" > hello.txt
```

- Taking standard input from hello.txt

```
$ tr [a-z] [A-Z] < hello.txt
```

- Input/output redirection

- Piping the standard output of echo to the standard input of tr:

```
$ echo "Hello World!" | tr [a-z] [A-Z]
```

- Redirecting standard output to standard error:

```
$ echo "Hello World!" >&2
```

- Redirecting the standard error to /dev/null:

```
$ echo "Hello World!" 2> /dev/null
```

- Input/output redirection

- Redirecting both the standard output and the standard error:

```
$ echo "Hello World!" >& /dev/null
```

- Piping both the standard output and the standard error:

```
$ echo "Hello World!" |& tr [a-z] [A-Z]
```

- Input/output redirection

- One of the most obscure features of bash are here documents.
- This type of redirection instructs the shell to read input from the current source until a line containing only a selected word (with no trailing blanks) is seen. All of the lines read up to that point are then used as the standard input for a command.
- E.G. using cat as a text editor:

```
$ cat > output.txt << __EOF__
Hopefully this is the worst text editor that you'll ever use.
__EOF__
```

- Filters

- Filters are programs that take their standard input and apply a transformation before outputting the result to their standard output.
- The most common filters are:
  - tr: translate and/or delete characters
  - cut: print selected parts of lines
  - sort: sorts lines
  - uniq: remove duplicate lines
- Check GNU coreutils documentation for more filters: https://www.gnu.org/software/coreutils/manual/coreutils.html

- Shell scripting

- The bash shell can read a text file containing commands and run those commands one by one. These files are called shell scripts.
- Saving a sequence of commands in a file and being able to let the shell repeat that sequence without intervention is a nice feature, but shell scripts allow conditional execution of commands based on the exit codes of other commands.
- Each process in a UNIX-like OS can set an integer value on exit. This value is relayed to the parent process on exit and thus is called exit code. 0 means success, everything else is considered a failure.

- Shell scripting - if

```
if command
then
block-of-commands
else
block-of-commands
fi
```

- Shell scripting – if (example)

```
if [[ $SHELL ~= 'bash$' ]]
then
echo "You are running a version of bash"
else
echo "You are not running bash"
fi
```

- Shell scripting – while loop

```
while command
do
block-of-commands
done
```

- Shell scripting – for loop

```
for VAR in SEQUENCE
do
block-of-commands
done
```

- Shell scripting – putting it all together

- Bash scripting can be used to automate tasks.
- The most basic way to do it is to come up with a sequence of commands that performs the task and put it all into a text file.
- Sometimes the task requires to repeat one (or more) commands a certain number of times (E.G. when scanning a list): use a for loop instead of repeating the command(s).
- Sometimes your script should handle a command failure: use the "if" conditional.
- Sometimes your script should wait until a condition is true: use a while loop.

- Shell scripting – putting it all together

- Task: write a script that displays the size of the files in a directory.
- Ignore the directories, report only the files.
- Use `du -h filename` to get the size of a file named `filename`.

- Shell scripting – putting it all together

```
for FILENAME in $(ls -1)
do
    if [[ -d "$FILENAME" ]] ; then
        continue
    fi
    du -h "$FILENAME"
done
```

- Thanks for your attention!

Thanks for your attention!

Any questions?